Curso:

AI Engineering

Módulo 8: Proyecto final

CODERHOUSE

Objetivos

1. Consolidar RAG, agentes y API en un único repositorio profesional.
2. Aplicar arquitectura limpia y desarrollo asíncrono de punta a punta.
3. Configurar y demostrar la observabilidad del sistema con trazas.
4. Documentar decisiones técnicas para una audiencia de ingeniería senior.
5. Dejar el proyecto ejecutable con una sola instrucción.

Agenda

1. El camino recorrido
2. La entrega final: qué construir
3. Break
4. La hoja de ruta de integración
5. Cómo se evalúa
6. Cierre del programa

01

El camino recorrido

Las capas del sistema

Lo que ya construiste

- **Cimientos asíncronos:** clientes robustos que manejan streaming y latencia. Módulo 1.
- **Orquestación declarativa:** cadenas LCEL con contratos de datos y resiliencia. Módulo 2.
- **RAG avanzado:** recuperación híbrida y metadatos para reducir alucinaciones. Módulos 3 y 4.
- **Inteligencia colectiva:** un supervisor que delega en agentes especializados. Módulos 5 y 6.
- **Robustez y despliegue:** observabilidad y escala con Redis y FastAPI. Módulo 7.

Las lecciones

Asincronía

No es paralelismo. El 90% del tiempo se va esperando red, no calculando.

RAG

No es subir PDFs. Es relevancia: chunking, búsqueda híbrida y re-ranking.

Agentes

No es una cadena más larga. Son ciclos, estado compartido y delegación.

El mapa del proyecto

¿De dónde sale cada pieza?

- **La base del repo:** las pre-entregas de los Módulos 6 y 7 son el punto de partida.
- **El cliente y las cadenas:** lo del M1 y el M2 queda adentro, refactorizado a async.
- **El conocimiento:** el RAG del M3 y el M4 pasa a ser la tool del agente investigador.
- **El razonamiento:** el grafo con supervisor del M5 y el M6 es el corazón del sistema.
- **La cáscara:** la API, Redis y las trazas del M7 envuelven todo lo anterior.

Momentos de la Clase:

Actividad colaborativa

Para pensar

Para recordar

Coder Tips

El capstone no es un proyecto nuevo: es el ensamblado y el refactor de lo que ya entregaste. Si arrancás de cero, hacés el doble de trabajo por el mismo puntaje.

Esta es tu carta de presentación como AI Architect.

Un sistema capaz de razonar, recordar y escalar.



02

La entrega final

¿Qué construir?

Las cuatro capacidades

📄 **Entrega:** repositorio de GitHub con el sistema completo en Python 3.12+.

- **Conocimiento específico:** el sistema accede a su base documental vía RAG.
- **Razonamiento multi-paso:** los agentes deciden, delegan y corrigen sus errores.
- **Interfaz profesional:** una API documentada, no un script de consola.
- **Trazabilidad completa:** se puede ver qué pasó bajo el capó en cada ejecución.

El alcance real

¿Qué entra y qué no?

- **Se evalúa con despliegue local:** Docker Compose o un script run.sh alcanzan.
- **No se pide CI/CD:** ni GitHub Actions ni pipelines. No suma ni resta en la rúbrica.
- **Tampoco infraestructura en la nube:** no hace falta desplegar en ningún proveedor.
- **Sí se pide:** que levante con una sola instrucción y arranque en una máquina limpia.
- **La lectura:** el foco está puesto en la arquitectura, no en el devops.

Errores a evitar

Lo que hunde una entrega final

- **Hard-coding:** IDs de índices de Pinecone o nombres de modelo escritos en el código.
- **La alternativa:** variables de entorno siempre, y un .env.example en el repo.
- **Sincronicidad escondida:** un requests adentro de una función async def.
- **La salida:** httpx o los SDK asíncronos oficiales de cada proveedor.
- **Falta de trazas:** un sistema que funciona pero es una caja negra. Eso no puntúa.

Momentos de la Clase:

Para recordar

Coder Tips

El error de sincronicidad es el más difícil de ver: un requests adentro de un async def no rompe nada, solo bloquea el event loop. Revisá los imports antes de entregar.

Material ampliado

Cuestionario de tarea

03

La hoja de ruta

Paso 1: la unificación

Un solo proyecto, una sola estructura

Objetivo: dejar de tener siete repos y pasar a tener uno.

Durante este paso vamos a:

- Mover la lógica de las pre-entregas del Módulo 6 y el 7 a una estructura unificada.
- Separar por capas: agentes, grafo, API, observabilidad y configuración.
- Consolidar las dependencias en un solo requirements.txt o pyproject.

Resultado esperado: un repo que se entiende sin conocer el historial del curso.

Paso 2: todo async

Nada de sync adentro de async

Objetivo: que toda la comunicación externa sea no bloqueante.

Durante este paso vamos a:

- Revisar cada llamada al LLM, a la vector DB y a Redis.
- Reemplazar requests por httpx y los SDK síncronos por sus versiones async.
- Confirmar que no queda ningún bloqueo escondido en el event loop.

Resultado esperado: el sistema sostiene peticiones concurrentes sin degradarse.

Paso 3: la memoria

Checkpointer y Pydantic

Objetivo: que el sistema no pierda nada y no confíe en nada.

Durante este paso vamos a:

- Implementar el Checkpointer de LangGraph para que las conversaciones sobrevivan.
- Validar con Pydantic todas las entradas y salidas de la API.
- Validar también los inputs y outputs de las herramientas de los agentes.

Resultado esperado: estado persistente y contratos de datos en cada frontera del sistema.

Paso 4: las trazas

Cinco pruebas con evidencia

Objetivo: demostrar que el sistema es observable, no solamente afirmarlo.

Durante este paso vamos a:

- Correr al menos cinco pruebas completas de punta a punta.
- Verificar que las trazas se visualizan bien en Arize Phoenix o LangSmith.
- Guardar la evidencia: capturas del dashboard con las trazas activas.

Resultado esperado: cada ejecución tiene su árbol de spans y su costo asociado.

Paso 5: el arranque

README, diagrama y un comando

Objetivo: que cualquiera pueda levantarlo sin preguntarte nada.

Durante este paso vamos a:

- Escribir el README con el diagrama de flujo del grafo de agentes.
- Documentar las decisiones de arquitectura para ingeniería senior.
- Dejar el proyecto ejecutable con Docker Compose o un run.sh.

Resultado esperado: la entrega final, lista para subir.

04

Evaluación

¿Cómo se evalúa?

Rúbrica sobre 100 · Aprobás con 70

- **Arquitectura y coherencia — 30%:** las capas de los módulos anteriores integradas en un flujo lógico.
- **Robustez técnica — 25%:** patrones async y await en Python 3.12 y validación con Pydantic.
- **Persistencia y observabilidad — 20%:** Checkpointers en LangGraph y trazado con herramientas externas.
- **Documentación y despliegue — 15%:** README claro, diagrama de arquitectura y facilidad de ejecución.
- **Entorno y estándares — 10%:** Python 3.12+, gestión de dependencias y variables de entorno.

El checklist final

Revisá esto antes de subir

- **Clone en limpio:** cloná el repo en otra carpeta y probá que arranque.
- **Sin secretos:** ninguna API key en el código ni en el historial de commits.
- **Un solo comando:** docker compose up o ./run.sh y el sistema levanta.
- **El diagrama:** el grafo de agentes dibujado en el README, no descrito en prosa.
- **Las capturas:** las trazas de las cinco pruebas, visibles en el repo.
- **Python 3.12:** declarado y respetado en las dependencias.

Momentos de la Clase:

Actividad colaborativa

Para pensar

Para recordar

Coder Tips

Si tuvieras que explicarle tu arquitectura a un ingeniero senior en cinco minutos, ¿por dónde empezarías? Esa respuesta es el primer párrafo de tu README.

¿Y ahora qué?

El cierre del programa

- **Lo que cambió:** dejaste de programar flujos rígidos y pasaste a diseñar ecosistemas.
- **Lo que sabés hacer:** gestionar la incertidumbre de la IA con estado persistente y validación.
- **Lo que podés medir:** latencia por percentiles, costo por token y calidad de la recuperación.
- **Lo que te diferencia:** un sistema con trazas y supervisión humana no es un proyecto escolar.
- **El siguiente paso:** este repo es tu portfolio. Contalo bien y llevalo a las entrevistas.

¡Gracias por estudiar con nosotros!

We have provided this presentation to you for informational and illustrative purposes on a confidential basis. We reserve the right to use of non-disclosure agreements (NDAs) to protect our privacy and intellectual property rights. The information contained in this document is highly sensitive, confidential and/or proprietary and is intended for the express use of the intended recipient, as denoted on the title page of this document. The recipient of this presentation agrees by its receipt not to reproduce, duplicate, or reveal, in whole or in part, information presented herein without written permission of Coderhouse. No representation or warranty, expressed or implied, is made as to the accuracy or completeness of the information contained in this presentation. Any quote contained herein is for estimation purposes. Estimates are based on current, known requirements. Actual estimates may change once project elements are finalized or negotiated. This document may contain confidential pricing information. All pricing is subject to change.

Copyright © 2026 Coderhouse. All rights reserved. Coderhouse and the Coderhouse logo are trademarks or registered trademarks of Coderhouse. in the U.S. and/or other countries. All other trademarks are the property of their respective owners.